

0221 오후 경과 보고

이정환

수정한 curve_fit func 수식 손으로 증명

i)

두 점을 지나는 직선의 방정식

$A(x_1, y_1)$ $B(x_2, y_2)$ 를 지나는 직선의 방정식

일반형식 $\Rightarrow y = ax + b$ 기울기 $a = \frac{y_2 - y_1}{x_2 - x_1}$

$$\Rightarrow y = \frac{y_2 - y_1}{x_2 - x_1} x + b \quad \dots \text{①} \leftarrow \text{점 A 대입}$$

$$y_1 = \frac{y_2 - y_1}{x_2 - x_1} x_1 + b \quad \dots \text{②}$$

$$\text{①} - \text{②} \quad y - y_1 = \left(\frac{y_2 - y_1}{x_2 - x_1} \right) (x - x_1)$$

$$\Rightarrow y = y_1 + \left(\frac{y_2 - y_1}{x_2 - x_1} \right) (x - x_1) \quad (\text{선형성분: 직선 방정식})$$

$$\rightarrow f(x_1) = y_1, \quad f(x_2) = y_2 \quad \text{성립함}$$

$$1) f(x_1) = y_1 + \left(\frac{y_2 - y_1}{x_2 - x_1} \right) (x_1 - x_1) = y_1 + \left(\frac{y_2 - y_1}{x_2 - x_1} \right) \cdot 0 = y_1$$

$$2) f(x_2) = y_1 + \left(\frac{y_2 - y_1}{x_2 - x_1} \right) (x_2 - x_1) = y_1 + y_2 - y_1 = y_2$$

$f(x_1) \rightarrow y_1$

$f(x_2) \rightarrow y_2$

x_1 을 넣었을때 y_1 을 지나가는 것을 확인

ii)

곡률 계산 (이차항)

$$a(x - x_1)(x - x_2)$$

곡률 = $f(x)$ 의 이차계수 (2차 미분)

$x = x_1, x = x_2$ 일때
0 얻기 성립함

$$f(x) = a(x - x_1)(x - x_2) + \text{직선}$$

1차미분
공분배 사용

$$f'(x) = a[(x - x_1) + (x - x_2)]$$

$$f'(x) = a(2x - x_1 - x_2) \quad \leftarrow \text{곡선의 기울기 (변화율)}$$

2차미분 (곡률)

$$f''(x) = \frac{d}{dx} [a(2x - x_1 - x_2)]$$

$\begin{cases} a > 0 \\ a = 0 \\ a < 0 \end{cases}$ 의 따라
곡선의 형태가 달라짐

$$f''(x) = 2a$$

(현상) 앞에서 만든 곡선에 추가된 곡률을 더하여 곡선을 만든다

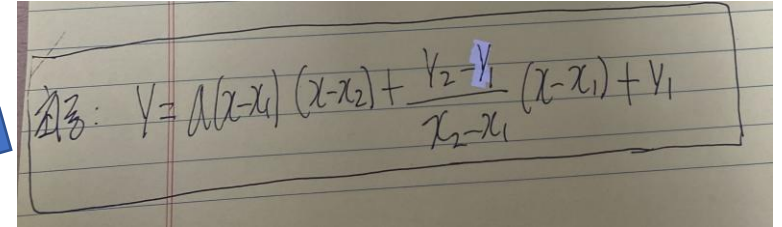
• $(x - x_1)(x - x_2)$ 는 곡률을 분할해주는 기본곡선

• a 는 그 곡선이 얼마나 휘어질지 3차항을 계산

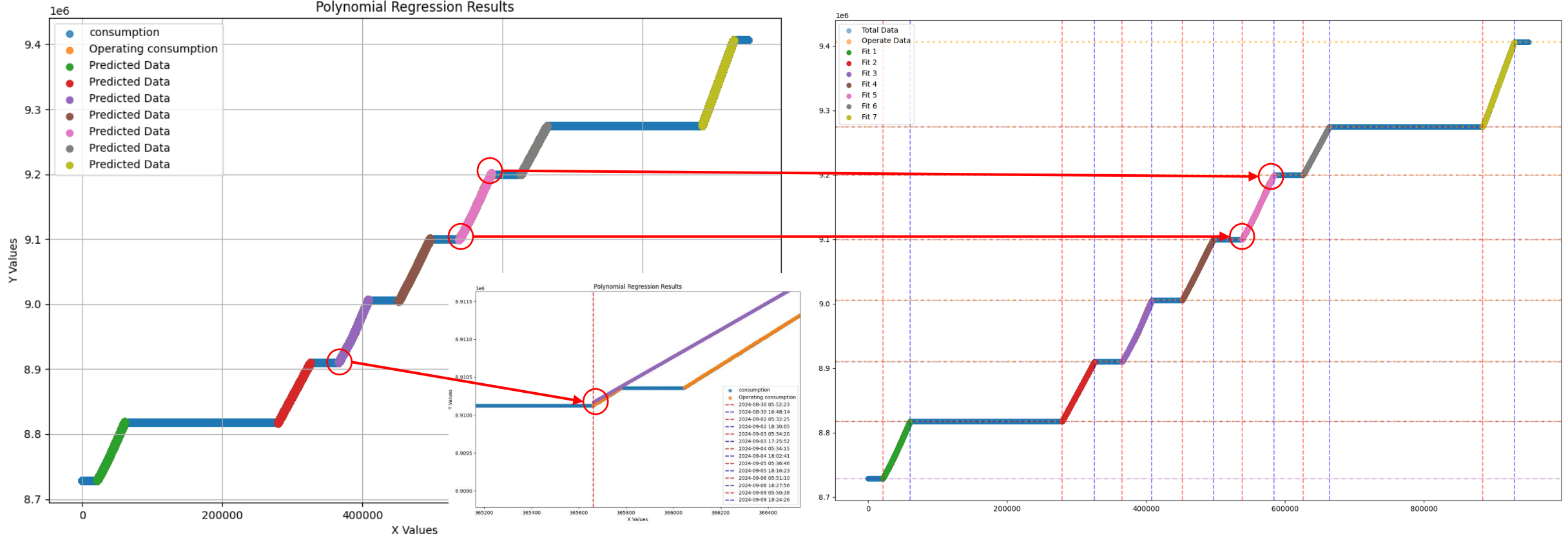
$$\text{최종: } y = a(x - x_1)(x - x_2) + \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) + y_1$$

수정된 최종 curve_fit func 코드

```
1 def curve_fit_poly_fitting(process_data, operate_data, collected_operate_slices, param, time_column='collect_time'):
2     fig, ax = plt.subplots(1, 1, figsize=(18, 12))
3     ax.scatter(process_data[time_column], process_data[param], label='Total Data', alpha=0.5)
4     ax.scatter(operate_data[time_column], operate_data[param], label='Operate Data', alpha=0.5)
5
6     popt_box = []
7     for idx, slice in enumerate(collected_operate_slices):
8         min_collected_operate_index = min(slice.index)
9         max_collected_operate_index = max(slice.index)
10
11         xdata = slice[time_column].to_numpy()
12         ydata = slice[param].to_numpy()
13
14         x_1 = slice.loc[min_collected_operate_index, time_column]
15         y_1 = slice.loc[min_collected_operate_index, param]
16         x_2 = slice.loc[max_collected_operate_index, time_column]
17         y_2 = slice.loc[max_collected_operate_index, param]
18
19         def func(x, a):
20             return a * (x - x_1) * (x - x_2) + y_1 + (y_2 - y_1) / (x_2 - x_1) * (x - x_1)
21             곡률 선형
22         popt, _ = curve_fit(func, xdata, ydata)
23
24         a_opt = popt[0] # 최적의 a 값
25         y_pred = func(xdata, a_opt)
26         popt_box.append((a_opt, x_1, x_2, y_1, y_2, xdata, y_pred))
27
28         ax.scatter(xdata, y_pred, label=f"Fit {idx+1}")
29         ax.axvline(x_1, color='red', linestyle='--', linewidth=1.5, alpha=0.5)
30         ax.axvline(x_2, color='blue', linestyle='--', linewidth=1.5, alpha=0.5)
31         ax.axhline(y_1, color='purple', linestyle='--', linewidth=1.0, alpha=0.5)
32         ax.axhline(y_2, color='orange', linestyle=':', linewidth=3, alpha=0.5)
33
34     plt.legend()
35     plt.show()
36     return popt_box
```


$$\text{공식: } Y = A(x-x_1)(x-x_2) + \frac{y_2-y_1}{x_2-x_1}(x-x_1) + y_1$$

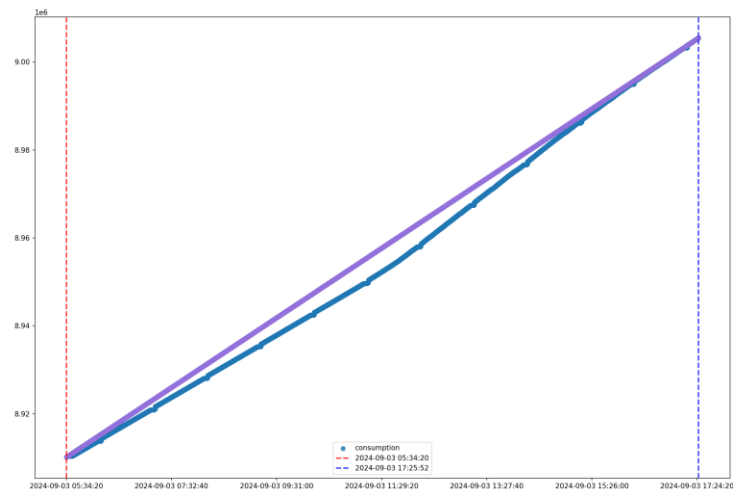
기존 2번 실험(2차 fitting)을 진행 했을 때 문제 해결 완료



- 기존 실험한 2차 fitting은 start, end 인덱스 또는 값이 시작과 종료 지점에 정확하게 맞지 않는 문제가 있었는데
- $f(x_{\min}) = y_{\min}$, $f(x_{\max}) = y_{\max}$ 2가지를 조건을 수식에 넣어서 값의 범위를 제한하여 이번 실험에서는 문제를 해결함
- 각 $x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$, 시작과 종료 시점에서 잘 맞는 것을 확인함

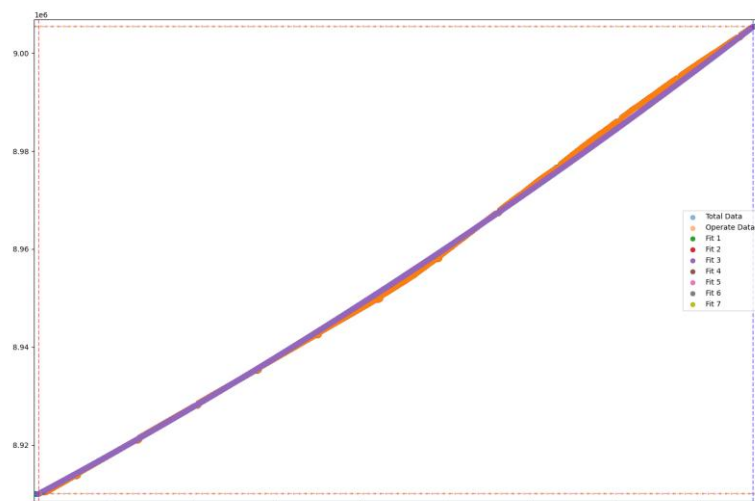
실험 별 기존 데이터와의 간격 차이 시각화

1번 실험
운전중 데이터 vs 선형보간



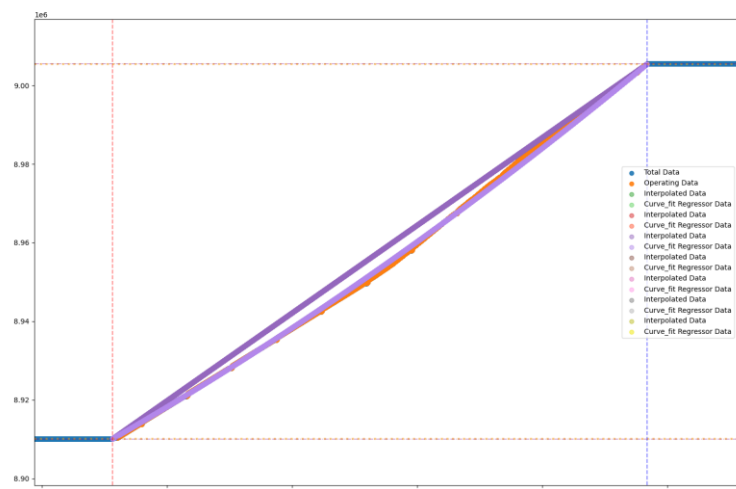
1번 실험 때 R-squared 가 가장 낮은 slice_3

2번 실험
운전중 데이터 vs 2차 fitting(curve_fit)

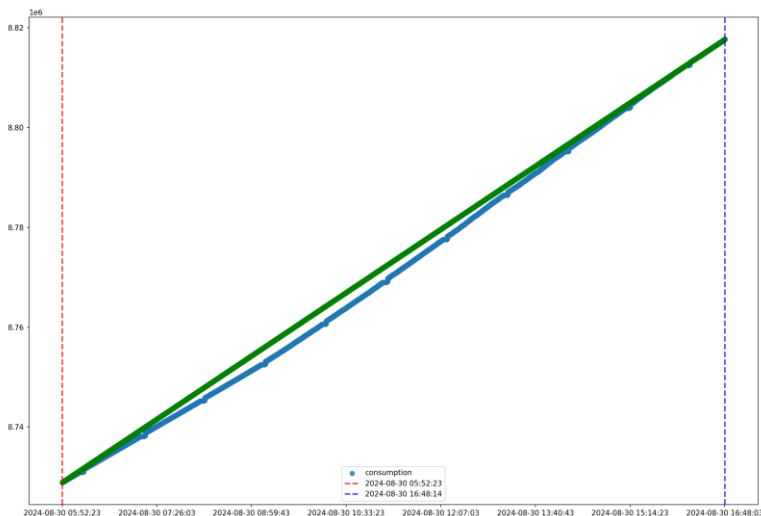


2번 실험에서 개선된 결과 slice_3

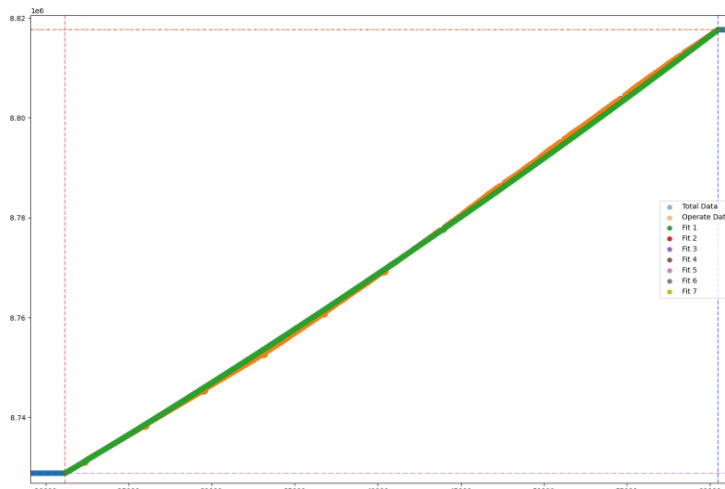
전체 시각화
운전중 데이터, 1번 실험, 2번 실험



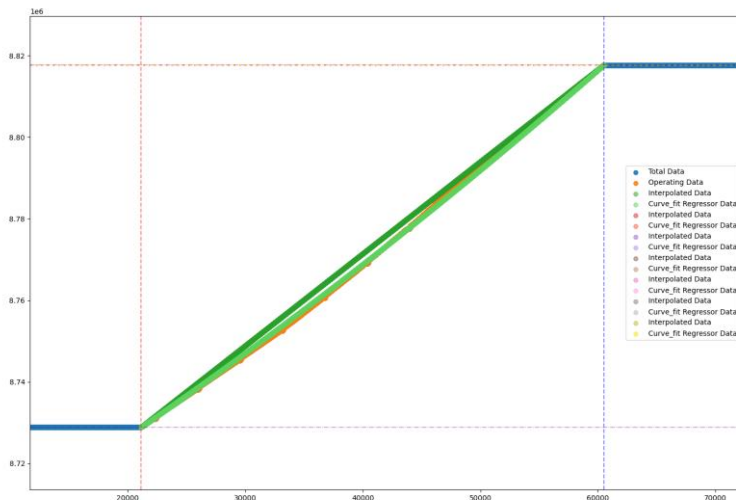
1번 2번 함께 시각화 slice_3



1번 실험 때 R-squared 가 두번째로 낮은 slice_1

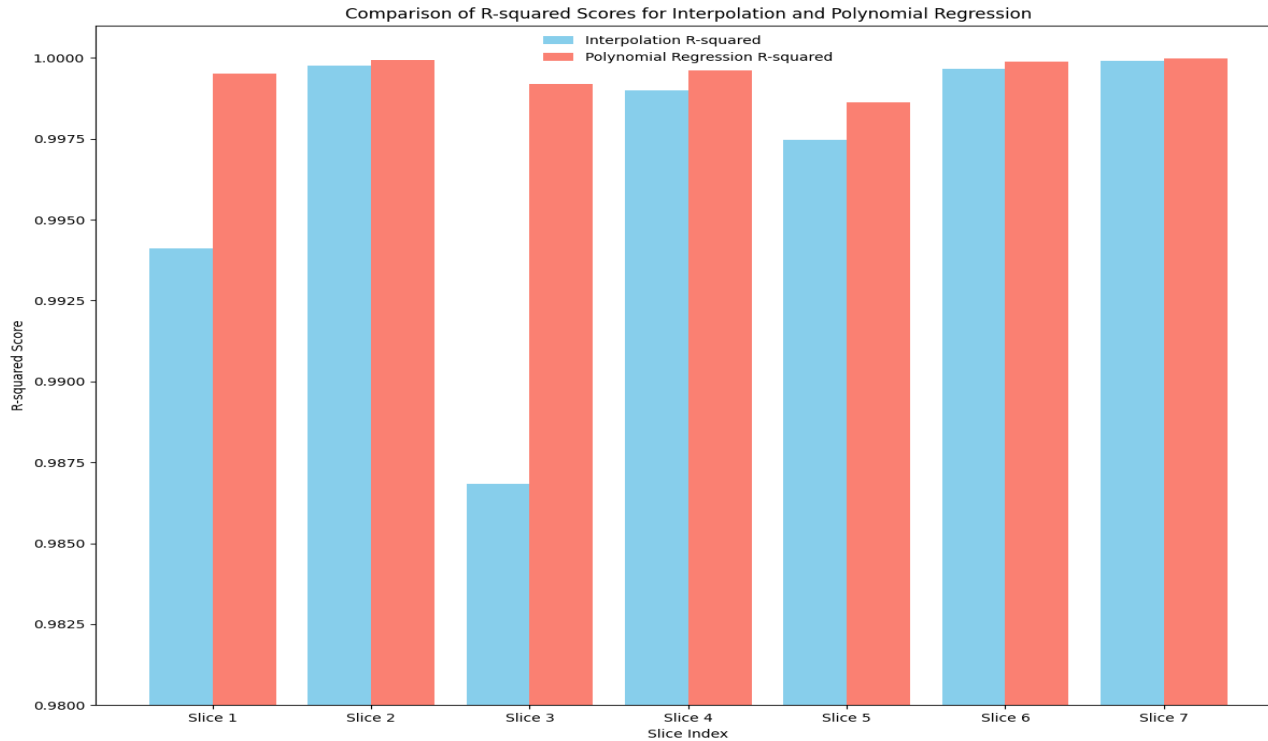


2번 실험에서 개선된 결과 slice_1



1번 2번 함께 시각화 slice_1

기존 데이터와 1번 실험과 2번 실험의 R-squared score 비교



- 1번 실험(두 인덱스를 linear interpolate 한) 결과보다 2번 실험(scipy optimize curve_fit)을 사용하여 예측 한 결과가 R-squared 점수가 더 높은 것을 확인 할 수 있었음
- 특히 1번 실험에서 예측 결과가 다른 것에 비해 낮았던 slice_3에서 크게 예측 성능이 좋아진 것을 확인 할 수 있었음

실험 1, 2의 R-squared 결과 비교

```
Slice 1: R-squared (Interpolation) = 0.994104, R-squared (Polynomial Regression) = 0.999510
Slice 2: R-squared (Interpolation) = 0.999762, R-squared (Polynomial Regression) = 0.999928
Slice 3: R-squared (Interpolation) = 0.986827, R-squared (Polynomial Regression) = 0.999199
Slice 4: R-squared (Interpolation) = 0.998997, R-squared (Polynomial Regression) = 0.999609
Slice 5: R-squared (Interpolation) = 0.997469, R-squared (Polynomial Regression) = 0.998623
Slice 6: R-squared (Interpolation) = 0.999665, R-squared (Polynomial Regression) = 0.999885
Slice 7: R-squared (Interpolation) = 0.999896, R-squared (Polynomial Regression) = 0.999980
```

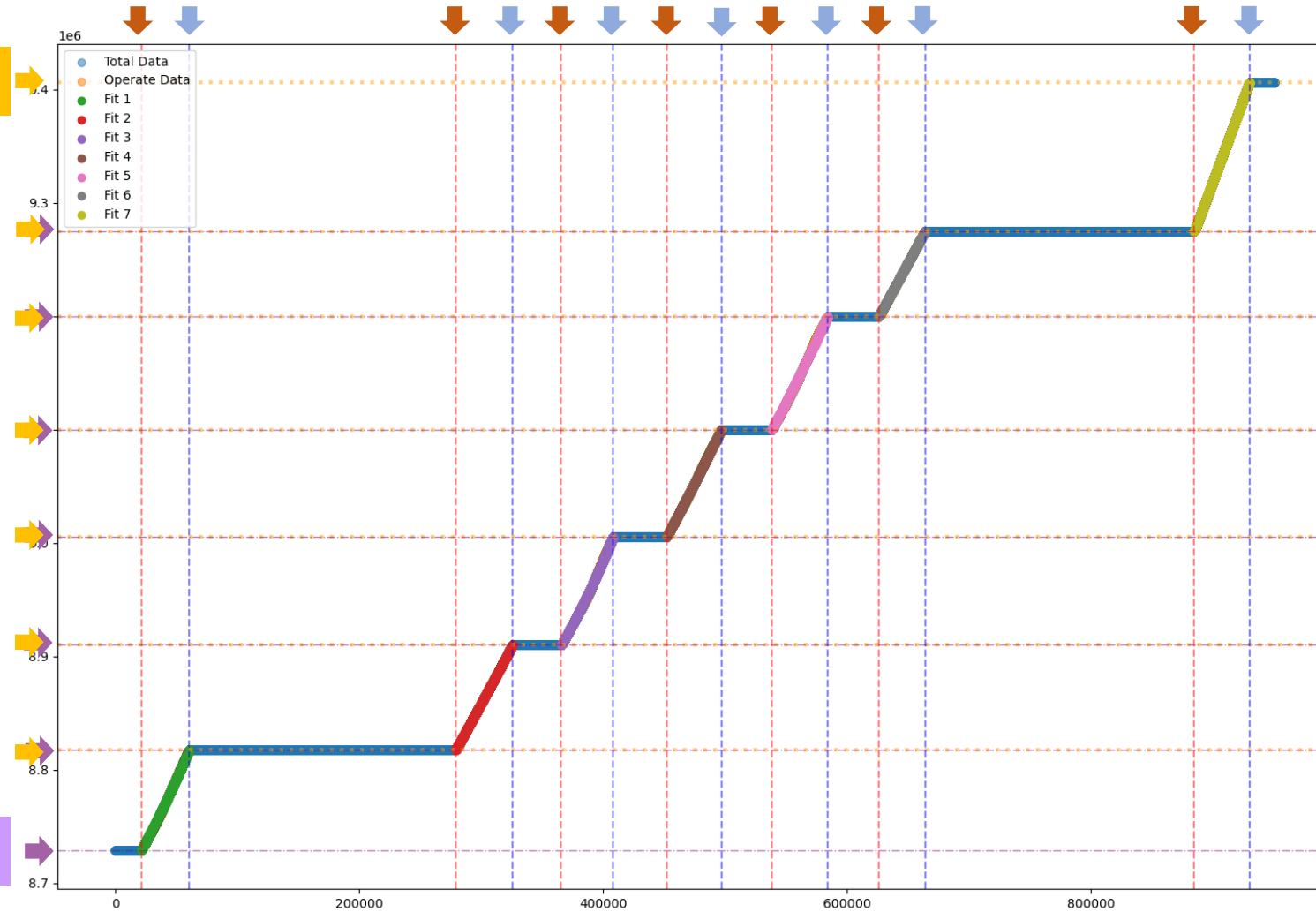
최종 결과 시각화

x_1: X_min

x_2: X_max

y_2: y_max

y_1: y_min



```
1 ax.axvline(x_1, color='red', linestyle='--', linewidth=1.5, alpha=0.5)
2 ax.axvline(x_2, color='blue', linestyle='--', linewidth=1.5, alpha=0.5)
3 ax.axhline(y_1, color='purple', linestyle='-.', linewidth=1.0, alpha=0.5)
4 ax.axhline(y_2, color='orange', linestyle=':', linewidth=3, alpha=0.5)
```